

Tree of Thoughts (ToT)

Módulo 6 · Ingeniería de Prompts

Exploración de múltiples caminos de razonamiento, BFS/DFS, evaluación de pensamientos y cuándo ToT supera a CoT.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Tree of Thoughts (ToT)

Chain of Thought sigue un único camino de razonamiento lineal: una secuencia de pasos que lleva de la pregunta a la respuesta. Para muchos problemas, esto es suficiente. Pero hay una clase importante de problemas donde el camino correcto no es obviamente el primero que se sigue, sino que requiere explorar múltiples alternativas, evaluar su viabilidad, y retroceder cuando un camino resulta ser incorrecto. Para estos problemas, Tree of Thoughts (ToT, Yao et al., 2023) propone una arquitectura de razonamiento más potente.

Tree of Thoughts convierte el proceso de razonamiento en una búsqueda en un árbol: en lugar de seguir un único camino, el modelo genera múltiples "pensamientos" candidatos en cada paso, evalúa su viabilidad, y expande solo los más prometedores. Es la diferencia entre correr en línea recta hacia donde crees que está la meta y explorar sistemáticamente el espacio de soluciones posibles. Para problemas de planificación, puzzles lógicos, y decisiones estratégicas con múltiples alternativas, ToT supera significativamente a CoT estándar.

La arquitectura de Tree of Thoughts

Los cuatro componentes del ToT

ToT tiene cuatro componentes. El descomponedor de problemas define cómo dividir el problema en pasos de razonamiento intermedios (pensamientos). El generador de pensamientos genera múltiples pensamientos candidatos para el siguiente paso (mediante sampling o propuestas secuenciales). El evaluador de pensamientos valora cada pensamiento candidato según su viabilidad como paso hacia la solución correcta. El algoritmo de búsqueda decide qué pensamientos expandir y cuáles descartar (BFS para exploración completa; DFS para exploración profunda con backtracking).

Sampling vs propuestas secuenciales

El generador de pensamientos puede usar dos estrategias. En el sampling, se generan múltiples pensamientos simultáneamente por muestreo independiente con temperatura alta: si se generan 5 pensamientos candidatos para el mismo paso, hay buena diversidad. En las propuestas secuenciales, el modelo genera pensamientos uno a uno con un prompt de propuesta que construye cada pensamiento sobre los anteriores, más adecuado para espacios de pensamiento más restringidos donde los pensamientos no son independientes entre sí.

Evaluación: ¿este pensamiento es prometedor?

El evaluador de pensamientos puede ser el mismo LLM con un prompt de evaluación ("¿Qué tan viable es este pensamiento como paso hacia la solución? Puntúa del 1 al 10") o un verificador externo (para problemas formales donde hay verificación objetiva). La auto-evaluación del LLM tiene limitaciones (los sesgos del modelo afectan la evaluación), pero en la práctica produce buenas señales de ordenación relativa de pensamientos.

Implementación práctica de ToT

La implementación completa de ToT con un LLM externo (como GPT-4 o Claude) usa múltiples llamadas al API en un loop de búsqueda: para cada nodo del árbol, llama al modelo para generar K pensamientos candidatos, llama al modelo de nuevo para evaluar cada candidato, selecciona los mejores candidatos para expandir según BFS o DFS, y repite hasta alcanzar la profundidad máxima o una solución satisfactoria.

El coste de ToT es alto: con K=5 pensamientos candidatos por nodo y profundidad D=3, se hacen $5 + 25 + 125 = 155$ llamadas al modelo para explorar el árbol completo. En la práctica, se usa beam search (mantener solo los B mejores pensamientos en cada nivel) para controlar el coste: con B=3 y D=3, son solo 9 expansiones y 9 evaluaciones = 18 llamadas al modelo, mucho más manejable.

Para muchos casos de uso empresariales, una versión simplificada de ToT que genera 3-5 enfoques alternativos para el problema y luego elige el mejor es suficientemente efectiva sin la complejidad completa del árbol: "Genera 3 formas distintas de abordar este problema. Para cada enfoque, identifica sus ventajas y limitaciones. Luego elige el enfoque más adecuado y desarrollalo."

SECCIÓN 4 · EJEMPLOS REALES

- Planificación estratégica con ToT simplificado: un consultor usa un prompt ToT para generar 4 estrategias distintas para resolver un problema de retención de clientes, evalúa cada una según criterios de coste, tiempo de implementación y impacto esperado, y selecciona la más adecuada para las condiciones específicas del cliente.
- Diseño de arquitectura de software: en lugar de que el LLM proponga una única arquitectura, se usa ToT para generar 3 arquitecturas alternativas (microservicios, monolito modular, serverless), evaluarlas según los requisitos técnicos y de negocio del sistema, y seleccionar la más apropiada con justificación explícita.
- Resolución de problemas de optimización de código: en benchmarks como HumanEval, ToT con evaluación de la corrección mediante ejecución de los tests produce mejores resultados que CoT solo, porque puede retroceder de soluciones incorrectas y explorar alternativas sistemáticamente.
- Toma de decisiones de inversión: generación de múltiples escenarios posibles (optimista, base, pesimista), evaluación de cada escenario según múltiples criterios, y síntesis de la recomendación considerando todos los caminos explorados.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar ToT para problemas que no requieren exploración de alternativas. Si el problema tiene una respuesta claramente correcta que se puede alcanzar con razonamiento lineal (cálculo directo, extracción de información, clasificación), ToT añade coste y complejidad sin beneficio.

Error 2: No controlar el coste del árbol. Sin beam search o límite de branching factor, el árbol puede crecer exponencialmente y producir cientos de llamadas al API por consulta. Define siempre un presupuesto de llamadas máximo antes de implementar ToT en

producción.

Error 3: Confiar ciegamente en la auto-evaluación del LLM. El modelo puede evaluar de forma inconsistente o sesgada, favoreciendo pensamientos que parecen más elaborados independientemente de su corrección. Cuando sea posible, usa evaluadores externos (tests automatizados, verificadores formales) en lugar de auto-evaluación.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para implementar ToT en producción: define el nivel de complejidad necesario. Para la mayoría de los casos de uso empresariales, la versión simplificada (generar K alternativas, evaluar, seleccionar) es suficiente y mucho más económica que el árbol completo. Usa frameworks como LangChain o LlamaIndex que tienen implementaciones de ToT simplificadas.

Para problemas que genuinamente requieren el árbol completo (planificación compleja, razonamiento simbólico, optimización con múltiples restricciones), implementa el ciclo de búsqueda explícitamente: función `generate_thoughts(estado, k)` que llama al LLM k veces para generar candidatos, función `evaluate_thought(estado, pensamiento)` que califica cada candidato, y la lógica de BFS/DFS que orquesta el árbol.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M6-T2 (CoT): ToT generaliza CoT añadiendo exploración de múltiples caminos de razonamiento.
- M10 (Agentes): los agentes usan arquitecturas similares a ToT para planificación con backtracking.
- M1-T4 (Sistema 2): ToT es la implementación más completa del pensamiento deliberativo Sistema 2.

SECCIÓN 8 · RESUMEN EJECUTIVO

Tree of Thoughts generaliza Chain of Thought convirtiendo el razonamiento en una búsqueda en árbol: generar múltiples pensamientos candidatos, evaluarlos, y expandir solo los más prometedores. Tiene cuatro componentes: descomposición del problema, generación de pensamientos (sampling o propuestas), evaluación (LLM o verificador externo), y algoritmo de búsqueda (BFS o DFS con beam search). Es la técnica correcta para problemas que requieren exploración de alternativas, planificación compleja, o backtracking. El coste es alto; controla con beam search. Para la mayoría de casos empresariales, la versión simplificada (generar K alternativas, evaluar, seleccionar) es suficiente.